The Reason for alias_method_chain

Suppose you have a module that defines a greet method. It might look like the following code.

part2/greet_with_aliases.rb
```
module Greetings
  def greet
    "hello"
  end
end

class MyClass
  include Greetings
end

MyClass.new.greet  # => "hello"
```

Now suppose you want to wrap optional functionality around greet—for example, you want your greetings to be a bit more enthusiastic. You can do that with a couple of *Around Aliases* (134):

```
class MyClass
  include Greetings

  def greet_with_enthusiasm
    "Hey, #{greet_without_enthusiasm}!"
  end

  alias_method :greet_without_enthusiasm, :greet
  alias_method :greet, :greet_with_enthusiasm
end

MyClass.new.greet  # => "Hey, hello!"
```

I defined two new methods: greet_without_enthusiasm and greet_with_enthusiasm. The first method is just an alias of the original greet. The second method calls the first method and also wraps some happiness around it. I also aliased greet to the new enthusiastic method—so the callers of greet will get the enthusiastic behavior by default, unless they explicitly avoid it by calling greet_without_enthusiasm instead:

```
MyClass.new.greet_without_enthusiasm  # => "hello"
```

To sum it all up, the original greet is now called greet_without_enthusiasm. If you want the enthusiastic behavior, you can call either greet_with_enthusiasm or greet, which are actually aliases of the same method.